

Technical University of Munich

School of Computation, Information and Technology
-- Informatics --

Bachelor's Thesis in Informatics

Self-Service Management of Time-Limited Access to Heterogeneous Resources

Self-Service-Verwaltung zeitlich begrenzter Zugriffe auf heterogene Ressourcen

Author:	Benjamin Schmitz
Examiner:	Prof. Dr. Stephan Krusche
Start Date:	01.10.2026
Submission Date:	31.03.2027

Abstract

A university research chair grants access to its compute hosts, chat workspace, and code repositories by hand, while the people who need that access change every semester. Requests travel by chat message or e-mail, nobody records an end date, and access granted for one project outlives the time a person spends at the chair. No single record states who holds which access or who approved it.

In this thesis, I develop a self-service platform on which every grant carries an expiry date from the moment somebody requests it. The platform organises all resources in one tree, provisions them through pluggable connectors, and lets maintainers decide on requests without administrative privileges. I evaluate the platform at the chair on correctness, administrative effort, and extensibility.

1 Introduction

A university research chair, or a team of similar size, runs shared infrastructure: Linux hosts with graphics processors that its members reach over Secure Shell (SSH), a Slack workspace, a GitHub organisation, and internal dashboards and databases. Thesis students, external collaborators, and working students join and leave this infrastructure every semester, on dates that the chair rarely tracks.

Keycloak[1] serves as the identity provider of the chair: it handles OpenID Connect login, groups, and profile attributes such as SSH public keys, and its Fine-Grained Admin Permissions scope administrative rights to selected groups. The Linux hosts read users and keys from Keycloak over the Lightweight Directory Access Protocol (LDAP), `pam_access` and `sudoers` decide who may log in or gain root rights, and an Ansible role configures each host.

Slack and GitHub manage membership through their own application programming interfaces (APIs) and tokens[2], and neither shares an administrative

[1] https://www.keycloak.org

[2] https://api.slack.com and https://docs.github.com/en/rest

interface with Keycloak. People ask for access to any of these systems by chat message or e-mail, and that message is often the only record of the change. The chair therefore holds identities in one place but grants access in three.

2 Problem

Requesters, such as thesis students and external collaborators, do not know who manages which system. A student who needs a host, a channel, and a repository asks several people and waits for each of them, and some bridge the delay with a shared login that hides who acted. Studies show that people bypass access control once asking costs more than a workaround [Bau+09, BSW08].

Maintainers, the supervisors who know whether a request is legitimate, cannot act on it, while the **administrators** who can act lack the context to judge it and become a bottleneck. The common workaround hands maintainers full administrative rights, which violates the principle of least privilege [SS75] and exposes every user and every host to a mistake made for a single project.

Access never ends. Nobody records when a person stops needing access, so access piles up, and former students keep login access to the compute hosts long after they leave, a decay that studies of enterprises report as well [Ker+02, SMJ01]. **Auditors** bear the consequence: the history lies scattered across systems and chat logs, and nobody can say who had access to a host last March and who approved it.

3 Motivation

Expiring access reverses the default of an organisation: access lasts until its expiry date unless its holder explains the need for more time. Requesters describe their project once and receive working access on the same day, maintainers decide about their own resources without privileges over anything else, administrators

work through a queue that maintainers have already reviewed, and auditors receive evidence as a by-product of daily operation.

Research on access control supplies the concepts for this vision. Role-based access control gives organisations a vocabulary for who may do what [San+96], temporal extensions bind roles to time [BBF01, Jos+05], attribute-based models express conditions [HKF15], and administrative and delegation models separate the right to decide from the right to assign [BS00, SBM99]. A platform that applies these concepts across systems without a shared administrative interface turns them into daily practice.

4 Objective

In this thesis, I develop a self-service platform on which requesters ask for time-limited access, maintainers decide, and access ends on its own; Figure 1 shows its actors and use cases as a Unified Modeling Language (UML) use case diagram. Following the object-oriented software engineering method of Bruegge and Dutoit[3], I pursue four objectives:

1. Unify heterogeneous resources in one tree.
2. Enforce an expiry date on every grant.
3. Delegate the approval of requests to maintainers.
4. Automate the recertification of active grants.

[3]https://pearson.com/store/p/object-oriented-software-engineering-using-uml-patterns-and-java/P200000003319

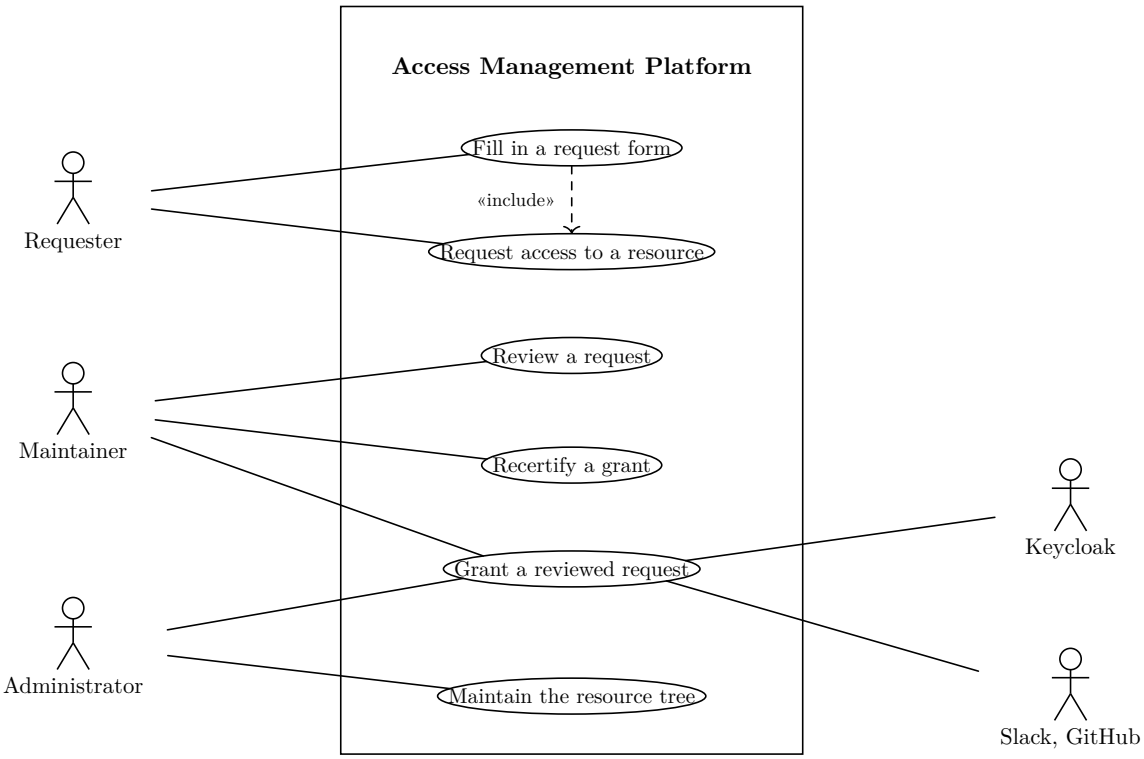


Figure 1: UML use case diagram of the access management platform. Requesters fill in request forms, maintainers review requests and recertify grants, and administrators grant reviewed requests and maintain the resource tree; a maintainer grants a reviewed request alone when it touches no binding that requires administrative privileges. Keycloak, Slack, and GitHub take part as secondary actors on the right, because a grant takes effect only when their state changes.

4.1 Unify Heterogeneous Resources in One Tree

I model the resources as a tree whose folders pass maintainership, audit policy, and grants down to leaves bound to target systems, as the UML object diagram in Figure 2 shows. One approval on a folder covers every resource beneath it, including later additions. A transitive-closure table stores the tree, and a fan-out cap keeps an approval on the root from provisioning the whole infrastructure.

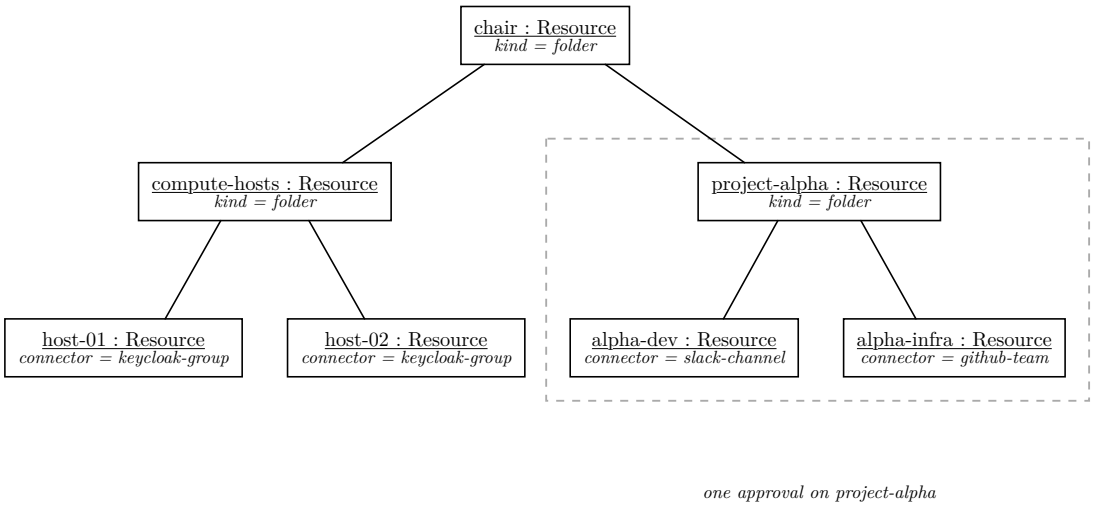


Figure 2: UML object diagram of a small resource tree at the chair. Folders such as `project-alpha` carry no binding of their own, while each leaf names the connector that provisions it. One approval on `project-alpha` covers both resources in the dashed box and would cover a third resource that an administrator adds to that folder later.

Every target system joins the platform through a connector that calls its API behind one contract: validate a binding, grant, revoke, and optionally list members. I implement the `keycloak-group`, `slack-channel`, `github-team`, and `webhook` connectors, whose credentials administrators configure at runtime, and Figure 3 shows how they fit into the platform. I reach this objective when the `webhook` connector, written after I freeze the framework, needs one new class and one registry entry.

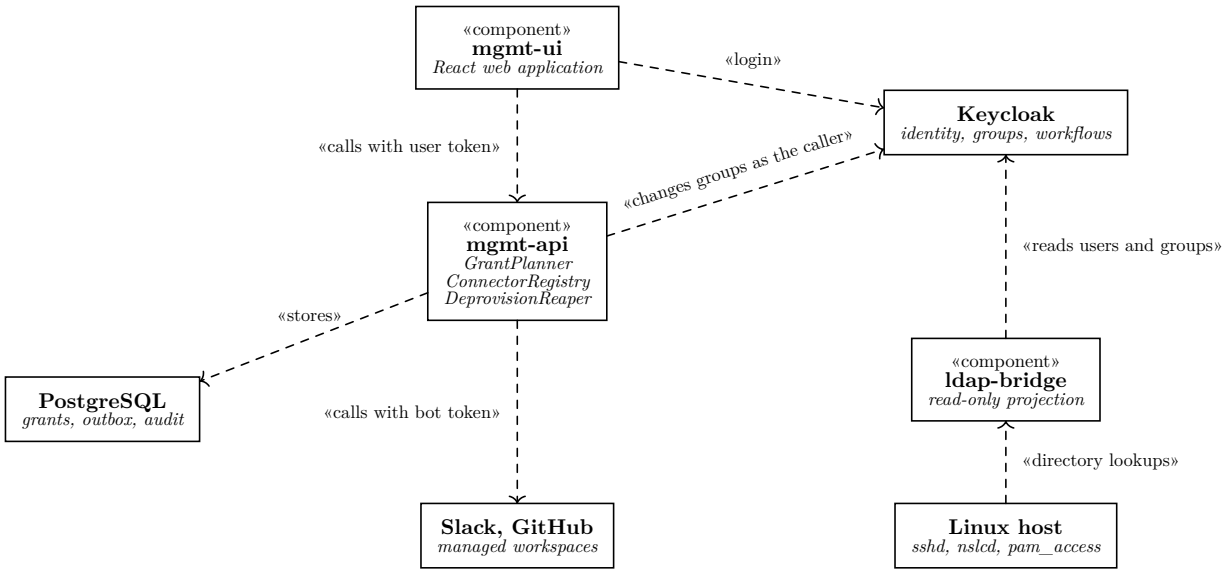


Figure 3: UML component diagram of the platform and its environment. The React web application `mgmt-ui` calls `mgmt-api`, which stores grants, the outbox, and the audit log in PostgreSQL and changes Keycloak only with the token of the calling user. The `ldap-bridge` component holds a separate read-only credential and projects users, groups, and SSH keys to the Linux hosts over LDAP, while the bot tokens for Slack and GitHub reach neither Keycloak nor the hosts.

4.2 Enforce an Expiry Date on Every Grant

The platform changes Keycloak only with the token of the calling user and holds no service account, so no background process of the platform can remove a membership. For Keycloak bindings, the platform therefore arms the workflow engine of Keycloak through its API at grant time. For all other bindings, a reaper without any Keycloak credential revokes expired grants through an outbox.

A drift check lists every membership that outlives its grant. I evaluate correctness with an automated scenario that provisions SSH, Slack, and GitHub access from one approval. I reach this objective when each access disappears after expiry and, in a second run with the workflow engine disabled on purpose, the drift check reports the remaining SSH membership.

4.3 Delegate the Approval of Requests to Maintainers

Requesters fill in a declarative request form instead of naming resources, and its rules turn answers into requests for a resource path, an access level, and a duration, which the platform routes to the maintainer of each resource. The server evaluates the rules, so a submission cannot request a resource that the form does not offer, and one submission covers everything that a new thesis student needs.

Keycloak gates its workflow API on a realm-wide administrative role that maintainers must not hold, so I split each decision into a review step, in which the maintainer records a decision without changing Keycloak, and a grant step, in which an administrator sets membership and expiry together. Slack-only requests need no grant step. I reach this objective when administrators perform fewer manual actions and requesters wait less than in the current process.

4.4 Automate the Recertification of Active Grants

Every resource carries a recertification interval that its descendants inherit, and each elapsed interval opens a campaign in which the maintainer keeps or revokes every grant with an immutable decision. Ignored campaigns escalate to an administrator and may revoke undefended grants automatically, but never when a campaign reached no maintainer, because a missing recipient signals a configuration error rather than a decision.

The platform writes every change to an audit log with the person who made it, and the admin event log of Keycloak keeps a second copy, because interactive changes carry the token of that person. I reach this objective when the platform answers, for any grant, who requested it and why, who reviewed and granted it, when it ended, and who confirmed it since.

5 Schedule

The thesis runs from 1 October 2026 to 31 March 2027. I divide the work into six iterations of four weeks, each ending with a vertically integrated increment that a user can try in the web application and that serves the objectives named in parentheses. I reserve the final two weeks, from 18 to 31 March 2027, as buffer for features that slip from earlier iterations.

- **Iteration 1, 1–28 October (Objective 1):** I deliver the Keycloak realm with OpenID Connect login, SSH key self-service, and the resource tree with inherited maintainers. Outcome: an administrator builds a tree and delegates one folder to a maintainer.
- **Iteration 2, 29 October – 25 November (Objectives 2, 3):** I deliver requests with a duration, the review and grant steps, the `keycloak-group` connector, the LDAP bridge, and the Ansible host role. Outcome: an approved SSH session stops working at expiry.
- **Iteration 3, 26 November – 23 December (Objectives 1, 2):** I deliver the connector contract, outbox, dispatcher, and reaper, the Slack and GitHub connectors, and their credentials screen. Outcome: one folder approval provisions a Slack channel and a GitHub team until expiry.
- **Iteration 4, 24 December – 20 January (Objective 3):** I deliver the form schema, the form builder with live preview, and server-side evaluation. Outcome: one onboarding submission yields requests across three connectors.
- **Iteration 5, 21 January – 17 February (Objectives 2, 4):** I deliver recertification campaigns with escalation, the drift report, and e-mail notifications. Outcome: an ignored campaign escalates, and the drift report lists a membership that outlives its grant.
- **Iteration 6, 18 February – 17 March (Objectives 1, 2):** I deliver the `webhook` connector and the deployment at the chair with the correctness scenario. Outcome: the connector needs one new class and one registry entry, and the scenario passes without the workflow engine.

Bibliography

[Bau+09] L. Bauer, L. F. Cranor, R. W. Reeder, M. K. Reiter, and K. Vaniea, "Real Life Challenges in Access-Control Management," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, ACM, 2009, pp. 899–908. doi: 10.1145/1518701.1518838.

[BSW08] A. Beautement, M. A. Sasse, and M. Wonham, "The Compliance Budget: Managing Security Behaviour in Organisations," in *Proceedings of the 2008 New Security Paradigms Workshop (NSPW)*, ACM, 2008, pp. 47–58. doi: 10.1145/1595676.1595684.

[SS75] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.

[Ker+02] A. Kern, M. Kuhlmann, A. Schaad, and J. Moffett, "Observations on the Role Life-Cycle in the Context of Enterprise Security Management," in *Proceedings of the 7th ACM Symposium on Access Control Models and Technologies (SACMAT)*, ACM, 2002, pp. 43–51. doi: 10.1145/507711.507717.

[SMJ01] A. Schaad, J. Moffett, and J. Jacob, "The Role-Based Access Control System of a European Bank: A Case Study and Discussion," in *Proceedings of the 6th ACM Symposium on Access Control Models and Technologies (SACMAT)*, ACM, 2001, pp. 3–9.

[San+96] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996, doi: 10.1109/2.485845.

[BBF01] E. Bertino, P. A. Bonatti, and E. Ferrari, "TRBAC: A Temporal Role-Based Access Control Model," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 191–223, 2001, doi: 10.1145/501978.501979.

[Jos+05] J. B. D. Joshi, E. Bertino, U. Latif, and A. Ghafoor, "A Generalized Temporal Role-Based Access Control Model," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 1, pp. 4–23, 2005.

[HKF15] V. C. Hu, D. R. Kuhn, and D. F. Ferraiolo, "Attribute-Based Access Control," *Computer*, vol. 48, no. 2, pp. 85–88, 2015, doi: 10.1109/MC.2015.33.

[SBM99] R. Sandhu, V. Bhamidipati, and Q. Munawer, "The ARBAC97 Model for Role-Based Administration of Roles," *ACM Transactions on Information and System Security*, vol. 2, no. 1, pp. 105–135, 1999, doi: 10.1145/300830.300839.

[BS00] E. Barka and R. Sandhu, "Framework for Role-Based Delegation Models," in *Proceedings of the 16th Annual Computer Security Applications Conference (ACSAC)*, IEEE, 2000, pp. 168–176. doi: 10.1109/ACSAC.2000.898870.

[UD26] S. Ucan and A. Demirel Ucan, "Generative AI in Student Writing: A Six-Level Taxonomy for Responsible Use in Higher Education Assessment," *Journal of Academic Ethics*, vol. 24, no. 2, p. 63, 2026, doi: 10.1007/s10805-026-09739-y.

Transparency in the Use of AI Tools

I used the following generative artificial intelligence (AI) tool while preparing this proposal. The level refers to the six-level taxonomy of generative AI use in student writing by Ucan and Demirel Ucan [UD26].

Tool	**Model / Version**	**Date(s)**	**Level**	**Purpose**	**Scope**
Anthropic Claude Code	Claude Opus 5.5 (claude-opus-5-5)	18.10.2026	5	Drafting text, authoring diagrams, researching and verifying references	Abstract, Sections 1-5, Figures 1-3, bibliography

I used Claude Code to draft the text of the abstract and of Sections 1 to 5, to author the three diagrams in Typst, and to search for the cited publications and verify their bibliographic data. I carefully checked all AI-generated text, revised it where necessary, verified every citation against the original publication, and compiled and inspected every diagram. I take full responsibility for the content of this proposal.